DAY 12 / 15 · Phase: Advanced · 26 October 2026

Oracles, Layer 2 rollups & bridges

Study guide by Manish Khadka · github.com/Khadka1996/Learning-Blockchain/tree/main/day-12

1. Learning objectives

- Explain what an **oracle** is and list the six checks a safe price reader makes.
- Explain how an **optimistic rollup** works and why a fraud proof needs only about $\log_2(n)$ rounds.
- Explain a **lock-and-mint bridge** and the three classic bugs (replay, duplicate signer, unbound message).
- Test defensively: boundaries, bad types, replay, and a differential fuzz against a JavaScript model.

2. The concept, step by step

A. Oracles: getting outside facts onto the chain

A smart contract cannot open a website. An **oracle** writes outside facts (like the ETH/USD price) onto the chain. It is like a weather board at a station: if the board is stale or wrong, everyone who trusts it dresses wrong. A lending app with a wrong price lends too much or force-sells people who were fine. Chainlink feeds publish the median of many nodes, updating when the price moves by a set percent (*deviation*) or after a fixed time (*heartbeat*).

Check in SafePriceReader	Catches
Sequencer up (L2)	Feed frozen while the L2 is down
Grace period	Mass liquidations right after a restart
answer > 0	Zero or negative price
0 < updatedAt ≤ now	Missing or future timestamp
age ≤ heartbeat	Stale price
min ≤ answer ≤ max	Glitch like $1,000,000 per ETH

Every failure **reverts** (fail closed): no price is better than a wrong price.

B. Layer 2 rollups: many transactions, one L1 entry

A rollup runs transactions off-chain and posts them to Ethereum in a **batch** with a claimed **state root** (a fingerprint of all balances). Like a bus instead of 100 taxis: the fixed fare is shared. In an **optimistic** rollup the result is believed unless someone proves fraud within a ~7-day window. The proof is a **bisection game**:

```
sequencer and watcher disagree on the final root of 1,000 txs
round 1: compare root after tx 500   -> differ  -> lie is in 0..500
round 2: compare root after tx 250   -> agree   -> lie is in 250..500
...
round 10: one tx left -> L1 re-runs just that tx -> who lied?
```

The cheating sequencer loses its **bond**; half goes to the watcher, half is burned (so it cannot "catch itself" for profit). ZK rollups instead prove correctness with math (Day 14).

C. Bridges: moving tokens between chains

A **lock-and-mint** bridge locks the token on chain A and mints an IOU on chain B, like a coat check. Validators sign the deposit; the destination mints when at least M of N *different* validators signed. Big hacks: Ronin 2022 (~$600M, 5 of 9 keys stolen), Wormhole 2022 (~$320M, signature check bypassed), Nomad 2022 (~$190M, any message accepted).

Bug	NaiveBridge	SafeBridge
Replay	Signs only (to, amount): reusable	Signs deposit id, marks it processed
Same signer twice	Counts signatures	Signers must be strictly increasing
Cross-chain replay	Nothing binds the chain	Digest has chainid + bridge address
Stolen keys ≥ threshold	Lost	Also lost: that is the trust model

3. Key code walkthrough

SafePriceReader.price(): the checks, in order

```
(, int256 answer, , uint256 updatedAt, ) = feed.latestRoundData();
if (answer <= 0) revert InvalidPrice(answer);            // 0 / negative = broken
if (updatedAt == 0 || updatedAt > block.timestamp)       // never filled, or "from the
future"
    revert InvalidTimestamp(updatedAt);
uint256 age = block.timestamp - updatedAt;
if (age > heartbeat) revert StalePrice(age, heartbeat);  // feed promised an update by
now
if (answer < minAnswer || answer > maxAnswer)           // circuit breaker band
    revert PriceOutOfBounds(answer);
return uint256(answer) * scale;                          // scale to 18 decimals
```

SafeBridge.mint(): count signers, not signatures

```
bytes32 digest = mintDigest(srcChainId_, depositId, to, amount); // binds chain, bridge,
id, to, amount
address last = address(0);
for (uint256 i; i < sigs.length; ++i) {
    address signer = ECDSA.recover(digest, sigs[i]);      // OZ rejects malleable (high-
s) sigs
    if (!isValidator[signer] || signer <= last) revert NotValidatorOrUnsorted(signer);
    last = signer;                                        // strictly increasing = no
duplicates
}
processed[depositId] = true;                              // effects before the external
call
token.mint(to, amount);
```

The evening fix: RollupL1.depositBond

```
const total = (this.bonds.get(sequencer) ?? 0) + (typeof amount === "number" ? amount :
NaN);
if (!Number.isSafeInteger(amount) || amount <= 0 || !Number.isSafeInteger(total))
  throw new Error("BadBond");   // no negative, NaN, 1.5, or text like "5"
```

4. Results from the run

Scenario	Naive	Safe
Feed silent for 2 hours	$2,000	REFUSED: StalePrice(7204, 3600)
Feed reports 0	$0	REFUSED: InvalidPrice(0)
Feed reports -1	CRASH (overflow)	REFUSED: InvalidPrice(-1)
Glitch $1,000,000	$1,000,000	REFUSED: PriceOutOfBounds
L2 sequencer down	$1,800	REFUSED: SequencerDown()

Txs in batch	L1 gas per tx	vs one L1 transfer (21,000)
1	47,128	2.2x more expensive
10	5,738	3.7x cheaper
100	1,597	13.1x cheaper
1000	1,183	17.8x cheaper

Fraud proof: a lie after tx #637 of 1,000 was found in **10 rounds**; the sequencer lost its 10 ETH bond, the watcher got 5 ETH. Bridge: the same 2 signatures replayed 5 times minted **500** tokens on the naive bridge for a 100-token lock; the safe bridge refused with `AlreadyProcessed`. Break-even batch size: **3** transactions.

5. QA & security findings

ID	Sev.	Finding	Fix
L1	Low	Rollup balances near 2^53 lose precision, making a fraud proof impossible	Genesis must be exact safe integers
L2	Low	Zero feed address gave an empty revert	Config checks run first
L3	Low	WrappedToken could be wired to address(0)	Constructor reverts
E1	Low	depositBond accepted -5, NaN, 1.5 and the text "5"	Positive safe integer only (BadBond)
P1–P5, E2–E6	Info	Fee-on-transfer, front-running, replay, self-challenge, 64-byte signatures, band edges, grace edge, Merkle duplication, window edge	All safe; tests added

Final: **56 / 56 tests**, ESLint + Solhint clean, `npm audit --omit=dev` 0 vulnerabilities, Slither 0 high / medium. Known issue: if M validator keys are stolen the bridge loses (trust model).

6. Common mistakes & interview questions

- Using one "stale" time for every feed instead of that feed's heartbeat.
- Signing only (to, amount) in a bridge, with no deposit id or chain.
- Counting signatures instead of different signers.
- Not checking the L2 sequencer feed on Arbitrum or Optimism.
- Trusting JavaScript numbers for money (negative, NaN, text, above 2^53).

1. What is an oracle and why is it a trust decision? It brings off-chain data on-chain; your contract is only as safe as the feed and how defensively you read it.

2. Why does an optimistic rollup take about 7 days to withdraw? That is the challenge window in which anyone can prove fraud. It is the security, not a bug.

3. How does bisection find one bad transaction in 1,000? Halve the disputed range each round by comparing state roots; after $\lceil \log_2 1000 \rceil = 10$ rounds L1 re-runs one transaction.

4. What must a bridge signature bind? Chain id, bridge address, source chain, deposit id, recipient and amount, then mark the deposit processed.

5. Optimistic vs ZK rollup? Optimistic assumes correct and allows fraud proofs; ZK attaches a validity proof, so withdrawals finalize once it is verified.

7. Exercises summary

Level	Task	Run
Easy	Compute $\lceil \log_2 n \rceil$ bisection rounds and check it against real challenges	npm run ex1
Medium	Find the batch size where a rollup beats a plain transfer (formula vs measured: 3)	npm run ex2
Challenge	A JavaScript twin of SafePriceReader, differential-fuzzed against the contract	npm run ex3

Cheat sheet

- Oracle read = sign, timestamp, age, band (+ sequencer on L2). Fail closed.
- Rollup = share one L1 fee; optimistic = fraud proof within ~7 days; ZK = validity proof.
- Bisection rounds = $\lceil \log_2 n \rceil$; L1 re-runs one transaction.
- Bridge = bind everything, mark deposit used, count different signers.
- Wrapped supply must never exceed locked tokens.

8. References

1. Chainlink, Using Data Feeds: https://docs.chain.link/data-feeds/using-data-feeds
2. Chainlink, L2 Sequencer Uptime Feeds: https://docs.chain.link/data-feeds/l2-sequencer-feeds
3. ethereum.org, Layer 2: https://ethereum.org/en/layer-2/
4. ethereum.org, Optimistic rollups: https://ethereum.org/en/developers/docs/scaling/optimistic-rollups/
5. Vitalik Buterin, An incomplete guide to rollups: https://vitalik.eth.limo/general/2021/01/05/rollup.html
6. ethereum.org, Bridges: https://ethereum.org/en/developers/docs/bridges/
7. EIP-4844, Blob transactions: https://eips.ethereum.org/EIPS/eip-4844
8. EIP-2098, Compact signatures: https://eips.ethereum.org/EIPS/eip-2098
9. SWC Registry, Signature malleability (SWC-117): https://swcregistry.io/docs/SWC-117
10. Rekt, Ronin Network: https://rekt.news/ronin-rekt/